

Energy-Aware Tuning of Pthreads and OpenMP Applications for Asymmetric Multicore Architectures

Felipe N. Laguna¹, Marcelo K. Moori¹, Hiago Mayk G. de A. Rocha³, and Antonio Carlos S. Beck^{1,2}

¹Institute of Informatics – Federal University of Rio Grande do Sul (UFRGS), Brazil

²Center for Embedded Devices and Research in Digital Agriculture (CEDRA), SENAI-RS, Brazil

³National Laboratory for Scientific Computing (LNCC), Brazil

Emails: ¹{fnlaguna, mkmoori, caco}@inf.ufrgs.br, ³mayk@lncc.br

Abstract—Asymmetric Multicore Processors (AMPs), composed of heterogeneous cores that implement the same architecture, offer significant power–performance trade-offs. However, identifying the optimal thread count for parallel workloads remains challenging due to their complexity, even in symmetric systems. While most existing autotuning tools focus on OpenMP, Pthreads-based applications, essential for system-level and performance-critical software, are often neglected in AMP research. This paper introduces *Caipiratuning*, a unified framework that employs exponential search to automatically determine near-optimal thread-to-core mappings for both Pthreads and OpenMP applications. Evaluated on an Intel Alder Lake processor using 22 benchmarks, *Caipiratuning* achieves average improvements of 16% in execution time, 29% in energy consumption, and 35% in EDP compared to the default Linux scheduler. These results are achieved while significantly reducing search overhead relative to exhaustive search.

Index Terms—Thread Throttling, Energy Efficiency, Asymmetric Multicore Processors, Pthreads, OpenMP

I. INTRODUCTION

Modern multicore processors, with their continuously increasing computational capability and growing number of cores, have driven significant advances in a wide range of application domains, including artificial intelligence, logistics, and bioinformatics [1]. However, these advances come at the cost of increased architectural complexity, such as the presence of heterogeneous cores and deep memory hierarchies, which significantly affect both the performance and energy consumption of parallel applications [2]. In shared-memory environments, the scalability of parallel applications is often constrained by several factors, including off-chip memory bandwidth saturation and synchronization overheads. As a consequence, employing the maximum available number of threads does not necessarily lead to optimal performance nor to reduced energy consumption, highlighting the importance of carefully tuning parallel execution parameters [3], [4].

Building upon these observations, several studies have proposed techniques for tuning the number of threads in parallel applications running on Symmetric Multicore Processors (SMPs) [1]. In contrast, only a limited number of recent works have addressed their asymmetric counterparts, namely Asymmetric Multicore Processors (AMPs) [2], [5], [6]. The inherent heterogeneity of AMPs, which combine cores with distinct microarchitectures and computational capabilities, naturally leads to execution imbalance among parallel application threads. This imbalance may significantly degrade performance and pose additional challenges to the design of effective performance and energy-aware tuning techniques for such architectures.

In addition to the aforementioned observations, existing strategies in the literature primarily focus on OpenMP applications [7]–[9], leaving a significant gap in the optimization of parallel applications implemented directly using Pthreads. Although OpenMP is widely adopted due to its high-level abstractions [10], [11], a broad class of performance-critical software (such as system-level applications, high-performance runtimes, scientific kernels, and legacy codes) continues to rely on Pthreads to achieve fine-grained control over thread creation, affinity, synchronization, and memory placement [12]–[14]. In these scenarios, OpenMP abstractions may be unavailable or may introduce additional overheads that limit optimization opportunities. Furthermore, Pthreads-based implementations frequently serve as the underlying execution layer for higher-level parallel frameworks, implying that optimization strategies at the Pthreads level can propagate benefits across multiple programming models.

Despite this relevance, energy- and performance-aware tuning techniques targeting Pthreads-based applications remain largely underexplored, particularly on AMPs. In this paper, we address this gap by proposing a unified strategy for tuning the number of threads in parallel applications implemented using either Pthreads or OpenMP. Our approach employs an exponential (doubling) search [15], [16] strategy to efficiently explore the thread configuration space, aiming for optimizing the application objectives: execution time, energy consumption, and Energy–Delay Product (EDP); by adjusting the number of threads assigned to performance cores (P-cores) and efficiency cores (E-cores) on Intel Alder Lake processors.

In a nutshell, the main contributions of this paper are:

- A comprehensive design space exploration (DSE) involving eleven parallel applications implemented using OpenMP and Pthreads, executed on an Intel Alder Lake processor, providing significant insights into the performance and energy behavior of parallel workloads on AMPs;
- A Framework for determining the optimal number of threads assigned to performance cores (P-cores) and efficiency cores (E-cores) when executing a given parallel application on the target processor. The proposed approach is applicable to both OpenMP and Pthreads-based implementations.

Our experimental results show that, when compared to baseline strategies, the proposed approach achieves average improvements of up to 16%, 29%, and 35% in execution time, energy consumption, and EDP, respectively, across the evaluated applications.

II. BACKGROUND AND RELATED WORK

Asymmetric Multicore Processors (AMP) integrate cores with distinct performance and energy characteristics, typically

combining high-performance (P) cores with energy-efficient (E) cores, each based on different microarchitectural designs [17]. This architectural paradigm was pioneered by ARM through its big.LITTLE technology and its successor, DynamIQ, which currently power the vast majority of modern mobile devices [1]. Recently, this design has also transitioned into the high-performance desktop segment. A representative example is the Intel Core i9-12900K Alder Lake processor, employed in our experiments (Section V). It integrates two core types: 8 P-Cores, which support Hyper-Threading (HT), and 8 E-Cores, organized into two clusters of four cores [18].

Achieving optimal performance on AMP systems requires careful selection of the thread count (and, consequently, the number of active P-cores and E-cores) to match the workload characteristics and maximize system resource utilization. Previous research has focused on microarchitectural design enhancements [19], operating system-level task scheduling policies [1], [20], [21], and improvements to OpenMP loop scheduling mechanisms [9]. In contrast, thread-count tuning on AMPs has received limited attention, with only a small number of studies addressing this problem [2], [5], [6].

It is worth to mention that, since optimizing the number of threads is a non-trivial problem, several studies have proposed and evaluated runtime and compile-time strategies targeting homogeneous multicore architectures, i.e., not AMPs [22]–[25]: Lee et al. [24] introduced Thread Tailor; Wang et al. [22] proposed NuCore; De Sensi et al. [25] presented an analytical approach; [23] proposed a reinforcement-learning-based method; Lorenzon et al. [7] introduced Aurora, a runtime autotuning framework that applies hill-climbing search to select the number of threads per OpenMP parallel region while optimizing performance, energy consumption, or energy-delay product (EDP). Similarly, Bari et al. [8] demonstrated that selecting an appropriate thread count and scheduling policy can reduce energy consumption by up to approximately 38%.

Regarding AMPs specifically, Moori et al. [5] proposed HyTuning, a framework based on exponential search that dynamically selects both the number of threads and the type of cores used during the execution of OpenMP applications. Subsequently, Moori et al. [6] introduced Mimir, a transparent runtime library that automatically determines the ideal number of threads for each OpenMP parallel region without requiring source code modifications or recompilation. Finally, Moori et al. [2] extended these efforts by proposing LOKI, a genetic-algorithm-based approach that jointly optimizes thread count and CPU frequency for parallel regions on AMP systems.

Our Contribution. To the best of our knowledge, this work presents the first unified thread-count tuning approach applicable to both Pthreads and OpenMP on AMPs. In contrast to previous studies, which are largely restricted to OpenMP-based runtimes, our approach enables thread-count tuning for low-level Pthreads applications, thereby extending its applicability to a broader class of parallel software.

III. CAIPIRATUNING

Caipiratuning is a framework for the automatic tuning of thread count and thread-to-core mapping for parallel applications executing on asymmetric multicore processors (AMPs). It aims to optimize a user-defined objective (execution time, energy consumption, or EDP). As illustrated in Fig. 1, the framework operates as follows: (i) it takes as **Inputs** the application name (*App*) and the metric to optimize (*Opt*), and

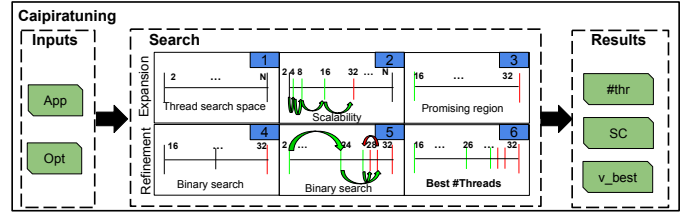


Fig. 1. Caipiratuning optimization flow.

identifies both the parallel programming model (e.g., OpenMP or Pthreads) and the characteristics of the target architecture; (ii) it then applies a heuristic **Search** strategy based on exponential search to efficiently explore the configuration space; and (iii) it **Results** the configuration that yields the best performance, defined in terms of the selected optimization objective, including the optimal number of threads and the corresponding thread-to-core mapping.

As previously mentioned, our approach employs a heuristic search strategy based on exponential search (see Fig. 1). Exponential search is a well-established technique for locating a target or optimal value in a totally ordered and potentially unbounded discrete search space [15], [16]. The algorithm proceeds in two phases: (i) an **Expansion** phase, in which the search interval is grown exponentially until a boundary condition is met (e.g., performance degradation); and (ii) a **Refinement** phase, in which the optimal configuration is identified within the identified interval.

During the **Expansion** phase, starting from an initial point x_0 , the algorithm evaluates the objective function at positions whose distance from x_0 grows exponentially, i.e., $x_0 + 2^0, x_0 + 2^1, x_0 + 2^2, \dots$, until a stopping condition is satisfied (steps 1-3). This condition typically occurs when the objective function ceases to improve or when a predefined constraint is violated. As a result, the expansion phase identifies a bounded interval $[x_L, x_U]$ that is guaranteed to contain the optimal solution.

In the **refinement** phase, a more precise search is performed within the interval $[x_L, x_U]$ to determine the optimal configuration. This phase is commonly implemented using binary search (in our case) or linear search, depending on the properties of the objective function.

To analyze the Exponential Search’s **computational complexity**, let k denote the position of the optimal solution in the ordered search space (steps 4-6). The number of evaluations required during the expansion phase grows logarithmically with respect to k , resulting in a complexity of $O(\log k)$. Similarly, when binary search is employed during the refinement phase, the number of additional evaluations is also $O(\log k)$. Therefore, the overall computational complexity of exponential search is $O(\log k)$ objective function evaluations.

Our approach is consistent with the observations of Moori et al. [5]. Accordingly, the **Expansion** and **Refinement** phases are executed multiple times – twice for single-objective optimization (execution time or energy consumption) and three times when optimizing for EDP – following different exploration orders. Specifically, the search traverses P-cores and E-cores in alternating sequences, either prioritizing high-performance cores (P→E) or energy-efficient cores (E→P), as illustrated in Fig. X.

Implementation Details. We implemented *Caipiratuning* in Python 3.10.12. It orchestrates application execution through system calls, automatically configuring the number of threads according to the selected optimization strategy. To support heterogeneous benchmark requirements, we employ a config-

uration dictionary that maps each application to its execution-specific parameters, including binary paths and command-line arguments. *Caipiratuning* dynamically constructs the execution command according to the underlying parallel programming model.

For **OpenMP applications**, concurrency is controlled via environment variables: `OMP_NUM_THREADS` specifies the number of threads, `OMP_PROC_BIND=TRUE` enforces thread affinity by preventing migration, and `GOMP_CPU_AFFINITY` explicitly binds threads to the selected set of P-cores or E-cores. For **Pthreads applications**, core affinity is managed externally using the `taskset` utility, which constrains the process execution to a predefined subset of cores. Some applications in Pthreads have a limit on the number of threads in their source code; we had to change the value of the limit to test these applications.

During the *Caipiratuning* optimization process, execution time is collected via and energy consumption is measured using the Intel Running Average Power Limit (RAPL) interface accessed through the Linux `perf` tool [26], [27]. These measurements are used to compute the corresponding EDP, which can then be used as the optimization objective.

To execute *Caipiratuning*, the user must provide the application binary to be optimized (required for Pthreads-based applications) and place it in the framework directory. Then, *Caipiratuning* will automatically identify a near-optimal number of threads and core allocation strategy, requiring minimal user intervention.

IV. METHODOLOGY

Experimental Environment. Experiments were conducted on an Intel Core i9-12900K (Alder Lake) [28] system with 8 P-cores (with Hyper-Threading) and 8 E-cores organized into two clusters. Each P-core has a private 1.25 MiB L2 cache, while each E-core cluster shares a 2 MiB L2 cache. All cores include 32 KiB L1 instruction and data caches, and the processor features a shared 30 MiB L3 cache. The system is equipped with 32 GB of RAM and runs Ubuntu 22.04 with Linux kernel 6.8.0.

Evaluated Benchmarks. We evaluated 19 kernels spanning diverse workload characteristics, including compute-bound, memory-bound, and I/O-bound behaviors. The implementations leverage both OpenMP and Pthreads. All kernels were compiled using GCC 11.4.0 with `-O3` flag enabled.

For **OpenMP**, we selected three kernels from the NAS Parallel Benchmarks (NPB) [29], version 3.0 (C implementation): Fourier Transform (FT), Embarrassingly Parallel (EP), and Scalar Penta-diagonal (SP). In addition, we included three kernels from the DAPHNE benchmark suite [30]: Euclidean Cluster (EC), NDT Mapping (NDT), and Points2Image (P2I).

For **Pthreads**, we evaluated one kernel from the FIO benchmark [12], two kernels from ParMiBench [13] (SUSAN and Basicmath), and nine workloads from CoreMark-PRO [14]: ZIP Compression (ZIP), XML Parsing (Parser), SHA-256 (SHA), CoreMark (Core), Linear Algebra (Linear), Radix-2 FFT (Radix), JPEG Compression (JPEG), Loops, and Neural Network (NNET).

Comparison Solutions. We evaluate *Caipiratuning* against four static mapping strategies: **Full cores(P→E)** (our baseline): use all cores, allocating threads from P cores to E cores; **Full cores(E→P)**: similar to the previous one, but allocating threads from E cores to P cores; **Full_P**: use only P

cores; **Full_E**: use only E cores; and **Exhaustive Search**: the best number of threads and thread-to-core mapping sequence achieved by evaluating all 24 possible configurations for each sequence (P→E and E→P).

V. RESULTS

Performance Analysis. We begin by comparing the performance of *Caipiratuning* against the evaluated strategies for each optimization metric. Fig. 2 presents the normalized results for (a) execution time, (b) energy consumption, and (c) EDP. All results are normalized against the default Linux scheduler behavior, which assigns threads from P to E cores (*Full cores P → E*), represented by the dashed line at 1.0; thus, lower values indicate better efficiency. Additionally, we evaluate the inverse mapping (*Full cores E → P*) to highlight that, in some cases, starting execution on E-cores can lead to better results due to execution variability and application behavior. The rightmost bars in each plot show the geometric mean (Geomean) across the benchmarks' kernels.

The results indicate that static strategies such as *Full P* and *Full E* are generally suboptimal. *Full E* can achieve competitive energy results in some cases; however, it may come at the cost of significantly degraded performance and EDP. Conversely, *Full P* tends to favor performance but leads to higher energy consumption and EDP. Overall, neither approach provides a balanced solution across all metrics, reinforcing that static mappings are not suitable as a general-purpose strategy.

In contrast, *Caipiratuning* consistently outperforms these static approaches, achieving near-optimal results across all evaluated metrics. It closely follows the behavior of the *Exhaustive Search* in most applications, significantly reducing EDP and energy consumption while maintaining or improving execution time compared to the default *P → E* policy (also *E → P*). As shown in the Geomean results, *Caipiratuning* reduces execution time by 16% (0.84 Geomean), energy consumption by 29% (0.73 Geomean), and EDP by 35% (0.65 Geomean) compared to the default *P → E* policy. Most importantly, it closely matches the optimal configurations found by the *Exhaustive Search*. Both methods converge to the same geometric mean for execution time, while the largest differences are observed in energy (5%) and EDP (1%).

It is important to note that in some cases *Caipiratuning* slightly outperforms the *Exhaustive Search*; this is attributed to natural execution variability. On the other hand, in specific workloads such as *radix*, *Caipiratuning* presents suboptimal results. This occurs because configurations with fewer threads (e.g., 2 threads) produced better outcomes than those explored later (e.g., 4 threads), preventing the heuristic from fully exploring more promising regions of the design space.

Convergence and Solution Quality. This section discusses the trade-off between search cost and solution quality. Static strategies – *Full cores*, *Full P*, *Full E*, *Full cores(P→E)*, and *Full cores(E→P)* – are computationally inexpensive as they require no exploration. However, by failing to explore the design space, they consistently produce the non-optimized solutions described in the previous section. *Caipiratuning* introduces a guided exploration strategy, allowing it to efficiently navigate the search space while maintaining a low overhead. This enables the method to identify high-quality configurations without requiring a full enumeration of all possible combinations.

The trade-off between optimality and efficiency becomes evident when comparing *Caipiratuning* against the *Exhaustive*

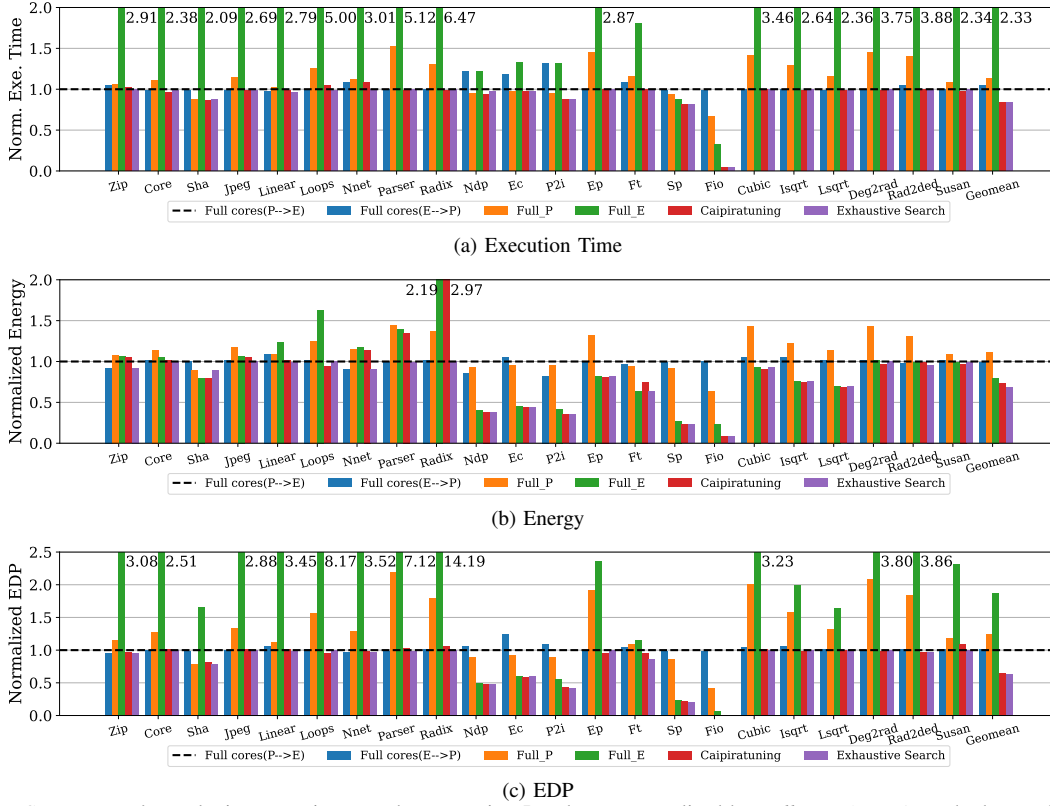


Fig. 2. *Caipiratuning*'s results in comparison to other strategies. Results are normalized by *Full cores(P→E)*, so the lower, the better.

Search. Since the *Exhaustive Search* evaluates every possible configuration (from 1 to 24 threads for each allocation sequence $P \rightarrow E$ and $E \rightarrow P$) and records all metrics simultaneously, its exploration mean time per application remains constant (≈ 256.34) regardless of the optimization goal. In contrast, *Caipiratuning* converges much faster to near-optimal results, as shown in Fig. 2.

Table I compares the time to find the solutions by *Caipiratuning* with those obtained via exhaustive search when optimizing for the EDP metric. It is interesting to observe in Figure 2.c that the geometric means values for *Caipiratuning* and exhaustive method are highly similar; the table shows that, despite arriving at nearly the same result, there is a significant time difference. It reports, for each evaluated application (rows) and both strategies (columns), the thread count, optimization time, and mapping sequence (Start Core – SC). Overall, *Caipiratuning* significantly outperforms the exhaustive method in search efficiency across all targets. For execution time, it reduced the search time by 81.3% (dropping to 47.98 minutes of mean per application). When optimizing for energy consumption, the search time was reduced by 67.9% (to 82.16 minutes of mean per application), and for EDP, it achieved a 50.1% reduction (to 127.79 minutes of mean per application). We can observe the difference in search time because, when starting with the E cores, we spend more time than with the P cores, and for the edp we execute the *Caipiratuning* twice, which results in even more time spent. These results demonstrate that *Caipiratuning* delivers high-quality solutions with a fraction of the overhead required by exhaustive methods.

TABLE I
DETAILED COMPARISON (SC = START CORE) WITH TIME IN MINUTES.

Alg	Caipira			Exhaustive		
	Num_t	Time(m)	SC	Num_t	Time(m)	SC
Zip	24	213.20	P	24	383.68	E
Core	24	146.49	P	24	268.01	P
Sha	17	63.62	E	16	115.78	P
Jpeg	22	178.82	E	24	324.58	P
Linear	22	83.65	E	23	149.96	P
Loops	24	184.08	P	24	346.54	P
Nnet	24	131.05	P	24	235.20	E
Parser	24	133.73	P	24	249.50	E
Radix	24	208.95	P	24	393.61	P
Ndt	8	32.63	E	5	121.47	E
Ec	8	35.67	E	8	102.05	E
P2i	1	10.78	P	1	136.63	P
Ep	24	52.45	P	24	96.67	P
Ft	10	40.58	P	9	98.76	P
Sp	8	186.47	E	5	713.09	E
Fio	1	4.27	E	1	429.39	E
Cubic	24	81.26	P	24	149.67	P
Isqrt	24	22.37	E	24	41.68	P
Lsqr	24	30.41	E	24	56.23	P
Deg2rad	24	450.17	E	24	829.49	P
Rad2deg	22	490.32	E	21	829.49	P
Susan	22	54.73	E	24	103.52	P

VI. CONCLUSION

This paper proposed *Caipiratuning*, a heuristic algorithm to find the best number of threads to run any parallel application on AMP Alder Lake processors. Our experimental results, considering twenty two applications, demonstrate that *Caipiratuning* can reduce the execution time, energy consumption and EDP compared to default parallel execution by an average of 16%, 29% and 35%, respectively.

ACKNOWLEDGMENT

This work was partially funded by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001, CNPq, FAPERGS Techfuturo - 23/2551-0002199-4, and by the Center for Embedded Devices and Research in Digital Agriculture (CEDRA) of SENAI-RS, with financial resources from the PPI IoT/Manufatura 4.0 / PPI HardwareBR of the MCTI, grant number 056/2023, signed with EMBRAPIL.

REFERENCES

- [1] J. C. Saez and M. Prieto-Matias, "Evaluation of the intel thread director technology on an alder lake processor," in *Proceedings of the 13th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2022, pp. 61–67.
- [2] M. K. Moori, H. M. G. D. A. Rocha, and A. C. S. Beck, "Energy-efficient execution of parallel regions on heterogeneous multi-cores," in *2025 38th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)*. IEEE, 2025, pp. 1–5.
- [3] J. Schwarzrock, H. M. G. d. A. Rocha, A. F. Lorenzon, and A. C. S. Beck, "Smoothing on dynamic concurrency throttling," in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2022, pp. 962–971.
- [4] J. Schwarzrock, H. M. G. de A. Rocha, A. C. S. Beck, and A. F. Lorenzon, "Effective exploration of thread throttling and thread/page mapping on numa systems," in *2020 IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, 2020, pp. 239–246.
- [5] M. K. Moori, H. M. G. d. A. Rocha, A. F. Lorenzon, and A. C. S. Beck, "Searching for the ideal number of threads on asymmetric multiprocessors," in *2023 XIII Brazilian Symposium on Computing Systems Engineering (SBESC)*. IEEE, 2023, pp. 1–6.
- [6] M. K. Moori, H. M. G. D. A. Rocha, A. F. Lorenzon, and A. C. S. Beck, "Efficient thread tuning for asymmetric multicore," in *2024 37th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)*. IEEE, 2024, pp. 1–5.
- [7] A. F. Lorenzon, C. C. De Oliveira, J. D. Souza, and A. C. S. Beck, "Aurora: Seamless optimization of openmp applications," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 5, pp. 1007–1021, 2018.
- [8] M. A. S. Bari, A. M. Malik, A. Qawasmeh, and B. Chapman, "Performance and energy impact of openmp runtime configurations on power constrained systems," *Sustainable Computing: Informatics and Systems*, vol. 23, pp. 1–12, 2019.
- [9] J. C. Saez, F. Castro, and M. Prieto-Matias, "Enabling performance portability of data-parallel openmp applications on asymmetric multicore processors," in *Proceedings of the 49th International Conference on Parallel Processing*, 2020, pp. 1–11.
- [10] T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Quantifying openmp: Statistical insights into usage and adoption," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2023, pp. 1–7.
- [11] A. F. Lorenzon, A. L. Sartor, M. C. Cera, and A. C. Schneider Beck, "Optimized use of parallel programming interfaces in multithreaded embedded architectures," in *2015 IEEE Computer Society Annual Symposium on VLSI*, 2015, pp. 410–415.
- [12] J. Axboe, "fio: Flexible i/o tester," <https://github.com/axboe/fio>, 2024, accessed: 2026-04-16.
- [13] S. M. Z. Iqbal, Y. Liang, and H. Grahm, "Parmibench-an open-source benchmark for embedded multiprocessor systems," *IEEE Computer Architecture Letters*, vol. 9, no. 2, pp. 45–48, 2010.
- [14] EEMBC, "Coremark-pro benchmark," <https://github.com/eembc/coremark-pro>, 2024, accessed: 2026-04-16.
- [15] J. L. Bentley and A. C.-C. Yao, "An almost optimal algorithm for unbounded searching," *Information processing letters*, vol. 5, no. SLAC-PUB-1679, 1976.
- [16] D. E. Knuth, *The Art of Computer Programming: Sorting and Searching, Volume 3*. Addison-Wesley Professional, 1998.
- [17] M. K. Moori, A. F. Lorenzon, H. M. G. d. A. Rocha, and A. C. S. Beck, "Gaamp: Automatic thread count/affinity and dvfs tuning for asymmetric multicore," in *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, vol. 1. IEEE, 2025, pp. 1–6.
- [18] C. Bilbao, J. C. Saez, and M. Prieto-Matias, "Flexible system software scheduling for asymmetric multicore systems with pmcsched: A case for intel alder lake," *CCPE*, p. e7814, 2023.
- [19] V. Dalakoti and D. Chakraborty, "Apple m1 chip vs intel (x86)," *EPRA International Journal of Research and Development (IJRD)*, vol. 7, no. 5, pp. 207–211, 2022.
- [20] B. Salami, H. Noori, and M. Naghibzadeh, "Online energy-efficient fair scheduling for heterogeneous multi-cores considering shared resource contention," *The Journal of Supercomputing*, pp. 1–20, 2022.
- [21] A. Fedorova, J. C. Saez, D. Shelepov, and M. Prieto, "Maximizing power efficiency with asymmetric multicore systems," *Communications of the ACM*, vol. 52, no. 12, pp. 48–57, 2009.
- [22] W. Wang, J. W. Davidson, and M. L. Soffa, "Predicting the memory bandwidth and optimal core allocations for multi-threaded applications on large-scale numa machines," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2016, pp. 419–431.
- [23] S. Malave and S. Shinde, "Reinforced manta ray foraging optimiser for determining the optimal number of threads in multithreaded applications," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 10, no. 3s, pp. 17–26, 2022.
- [24] J. Lee, H. Wu, M. Ravichandran, and N. Clark, "Thread tailor: dynamically weaving threads together for efficient, adaptive parallel applications," in *Proceedings of the 37th annual international symposium on Computer architecture*, 2010, pp. 270–279.
- [25] D. De Sensi, M. Torquati, and M. Danelutto, "A reconfiguration algorithm for power-aware parallel applications," *ACM Trans. Archit. Code Optim.*, vol. 13, no. 4, dec 2016.
- [26] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "Rapl in action: Experiences in using rapl for power measurements," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 3, no. 2, Mar. 2018. [Online]. Available: <https://doi.org/10.1145/3177754>
- [27] Linux perf Wiki, "perf: Linux profiling with performance counters," 2026, accessed: 2026-04-17. [Online]. Available: <https://perfwiki.github.io/main/>
- [28] Intel Corporation, "Intel core i9-12900k processor specifications," 2026, accessed: 2026-04-17. [Online]. Available: <https://www.intel.com.br/content/www/br/pt/products/sku/134599/intel-core-i912900k-processor-30m-cache-up-to-5-20-ghz/specifications.html>
- [29] NASA Advanced Supercomputing Division, "Nas parallel benchmarks," <https://www.nas.nasa.gov/software/npb.html>, 2024, accessed: 2026-04-16.
- [30] L. Sommer, F. Stock, L. Solis-Vasquez, and A. Koch, "Daphne-an automotive benchmark suite for parallel programming models on embedded heterogeneous platforms: work-in-progress," in *Proceedings of the International Conference on Embedded Software Companion*, 2019, pp. 1–2.